This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Focal Plane Visual Feature Generation and Matching on a Pixel Processor Array

Hongyi Zhang[1] Laurie Bose[2] Jianing Chen[2] Piotr Dudek[2] Walterio Mayol-Cuevas[1,3]
[1]University of Bristol [2]The University of Manchester [3]Amazon

Abstract

Pixel Processor Arrays (PPAs) are vision sensors that embed data and processing into every pixel element. PPAs can execute visual processing directly at the point of light capture, and output only sparse, high-level information. This is in sharp contrast with the conventional visual pipeline, in which whole images must be transferred from sensor to processor. This sparse data readout also provides several major benefits such as higher frame rate, lower energy consumption and lower bandwidth requirements. In this work, we demonstrate generation, matching and storage of binary descriptors for visual keypoint features, entirely upon PPA with no need to output images to external processing, making our approach inherently privacy-aware. Our method spreads descriptors across the memory of multiple pixel processors, allowing significantly larger descriptors than prior pixel-processing works. These larger descriptors can be used for a range of tasks such as place and object recognition. We demonstrate the accuracy of our in-pixel feature matching up to ∼94.5%, at ∼210fps, across a range of datasets, with a greater than 100× reduction in data transfer and bandwidth requirements over traditional cameras.

1. Introduction

The Pixel Processor Array (PPA) is a general-purpose programmable visual sensor. Standard camera sensor pixels can only capture the intensity of incoming light, while the "pixel processors" of PPAs can also store data in local memory, perform computation, and communicate with neighbouring processors. This massively parallel array of pixel processors enables a wide range of visual tasks to be performed entirely "in-pixel", promising improved system power consumption and latency. However, it also requires new efficient methods to be designed specifically for this novel sensor architecture [9]. In this paper, we propose an approach to compute, store, and match long binary feature descriptors entirely in-pixel. We envision such a system could be used in a wide range of applications, such as SLAM, Localisation and Recognition tasks. Crucially, our approach can be performed without images, or even descriptors, ever leaving the sensor's focal plane. We anticipate such a capability will be vitally important for preserving privacy in a world where camera-equipped, internet-connected devices such as VR/AR headsets and robotic vacuum cleaners are becoming commonplace. These have the potential to capture live footage of users' homes, but our approach eliminates this risk by ensuring that such data never leaves the sensor. Additionally, image transfer off-sensor is a slow, power-hungry operation, due to the sheer amount of data movement involved. By avoiding this and utilizing pixel processing, our approach becomes very power and bandwidth efficient compared to the standard visual pipeline in which captured images are continuously relayed from camera to processor. Our contributions are as follows.

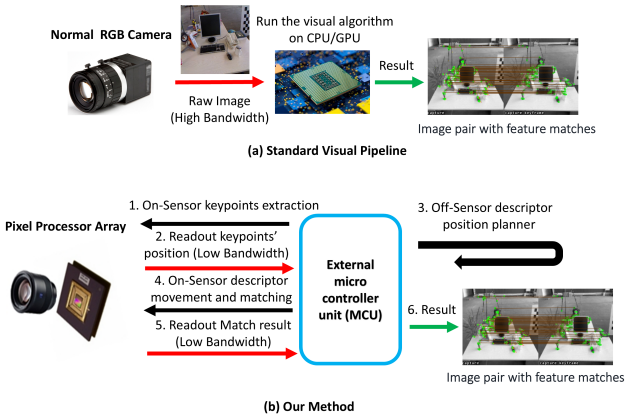

Figure 1. Standard visual pipeline (top) vs our approach (bottom). Red arrows indicate essential data exchange. Straight black arrows represent commands sent from the Microcontroller unit (MCU) to the PPA. The circular black arrow represents the program running in the MCU. The green arrows represent the output of matchings. Our system runs the steps in the order shown by the numbers.

1. **In-Pixel Descriptor Storage** Our approach enables the use of significantly longer (128 & 256 bit) "in-pixel" descriptors than have been demonstrated in prior PPA-based works. This is achieved by storing descriptors across multiple pixel processors to mitigate the reduced memory available within a single processor.
2. **In-Pixel Descriptor Matching** We demonstrate matching of descriptors that are stored entirely in-pixel, across

multiple pixel processors. This allows feature associations to be determined with neither images nor descriptors ever leaving the sensor. Two methods for determining such matching are proposed, one using digital operations, and a second exploiting specific analogue capabilities of the SCAMP PPA used in our experiments.

3. **Implementation & Evaluation Upon a PPA** We implement and demonstrate our approach upon the SCAMP PPA [6]. Specifically, we detect keypoints, generate corresponding descriptors, and use these descriptors to match against previously seen keypoints from a stored keyframe. We evaluate such keypoint matching on two key tasks and datasets: i) perspective and image change with the Oxford Affine Covariant Feature dataset [16, 21] and ii) localisation with the Oxford Robotic Car dataset [13, 14]. We also provide a breakdown of our system's computational costs and bandwidth usage.

2. Related work

2.1. Tracking, Mapping and Re-localization on PPA

The first in-pixel Visual Odometry system for PPAs was proposed by [2], tracking scale-less 4-DoF motion at over 1000Hz. This work estimated motion via an image alignment method, which comprises three transformations (Rotation, Shifting, and Scaling). Only data relating to the transformations that achieve alignment leave the sensor, from which ego motion is then derived.

The BIT-VO system on PPA [18] is designed for 6-DoF Visual Odometry tracking, with an architecture similar to ORB-SLAM [17]. Here, the PPA camera extracts keypoints, and a 256×256 binary edge image. These are both outputs from the PPA to be processed by an external PC. This data transfer bottlenecks performance to around 300Hz.

A PPA-based SeqSLAM system was proposed by [7], storing an image sequence in the layout of a snake path for pictorial mapping. Each image in the sequence is downsampled to just 8×8 pixels. Location is estimated via matching the images observed in the latest sequence against the image sequences of the "map". However, this approach only localises upon a single prefixed path.

2.2. Other Neuromorphic Sensors

PPAs are one form of neuromorphic sensor, however by far the most prevalent is the event camera. Instead of images, these sensors output the pixel coordinates of intensity changes in the form of event streams. This reduces the sensor data bandwidth, but shifts computational effort further down the visual pipeline, as significant computation is required to interpret these events [10–12, 15, 19, 20]. In contrast, the PPA can extract high-level information entirely by itself via in-pixel computation.

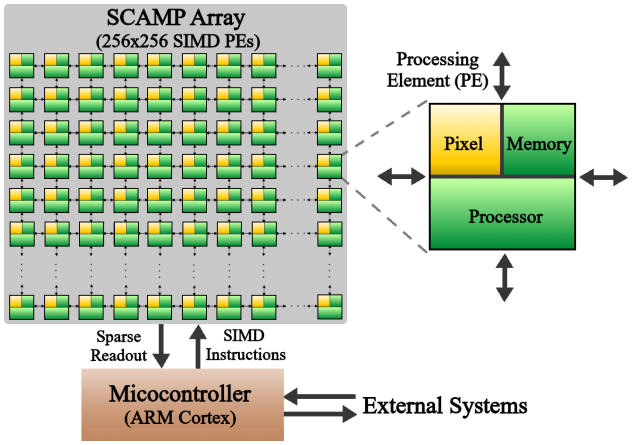

Figure 2. Overview of the SCAMP system. The sensor plane consists of a rectangular array of 65,536 pixel-processors (PEs). These PEs execute SIMD instructions sent from a connected ARM micro-controller.

2.3. SCAMP PPA Architecture

In this work we make use of the SCAMP system as shown in Figure 2, which at the time of writing is the most feature-rich PPA architecture. It features a 256×256 array of pixel processors or "Processing Elements" (PEs). Each PE has local light capture, processing, and memory. Specifically, each PE has 22 local digital registers (1-bit) and 6 local analogue registers (continuous values). A PE can perform a range of operations between these local memory registers, such as bitwise logic between digital registers, and addition/subtraction between analogue registers. This array of 65,536 PEs operates as a Single Instruction Multiple Data (SIMD) architecture, whereby a central micro-controller broadcasts instructions to the array, with all PEs then executing the same instructions in parallel. Images are captured directly into this array, each PE capturing one pixel of the whole image. Importantly, there is communication between neighbouring PEs, allowing data to be moved across the array, and for each PE to access visual data beyond its own captured pixel. Individual PEs can be masked to enable conditional execution of the broadcast instruction stream. A full list of SCAMP functions can be found at [1].

3. Binary Descriptor Generation & Storage

This section covers our in-pixel generation and storage of binary descriptors for keypoints. An existing implementation of the FAST corner detector for SCAMP [8] is used to detect keypoints. Binary descriptors are then generated for all PEs containing keypoints in parallel via pixel-processing. Each individual PE has extremely limited digital memory (22 bits), and for this reason our approach spreads descriptor storage across multiple PEs horizontally, one bit per PE. This allows us to use descriptors up to 256

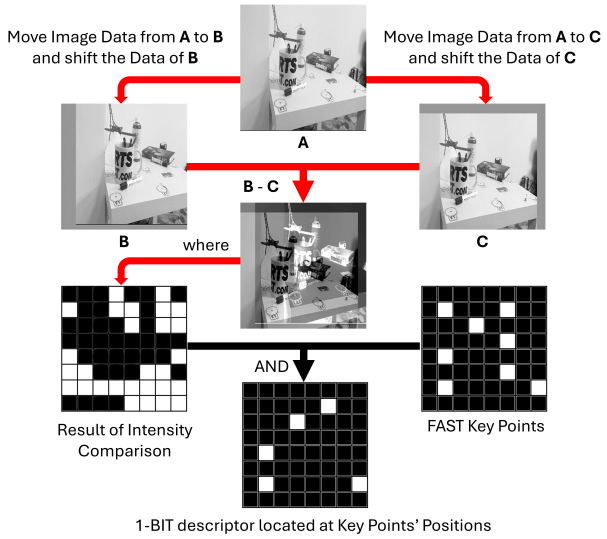

Figure 3. BRIEF descriptor generation process on PPA, generating 1 bit of each keypoint's descriptor from one point pair. Red/Black arrows represent analogue/digital operations. *where* finds out the pixels whose values are greater than 0.

bits in length, at which length each descriptor will occupy an entire row of PEs, as illustrated in Figure 5. The binary descriptor used here is BRIEF [5], however our approach can generalise to any binary keypoint descriptor. We now describe the generation and storage of these descriptors in detail.

3.1. In-Pixel Descriptor Generation

The BRIEF descriptor for a keypoint is generated from a number of intensity (*i.e.* brightness) comparisons between pairs of pixels around the keypoint. These pairs are chosen according to a specific sampling pattern. Each of these comparisons generates 1 bit of the descriptor. Specifically, the i-th bit of a descriptor is given by Equation 1, where p_1^i and p_2^i denote intensities of the i-th pair of pixels.

$$\text{descriptor } i^{th} \text{ bit} = \begin{cases} 1 & p_1^i > p_2^i \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

Crucially, each PE in SCAMP can only perform operations upon its own local data registers. This means a PE cannot perform a comparison between two pixels located in different PEs. Instead, that pixel data must be copied into the PE performing the comparison. This is achieved by "shifting" copies of the pixel data, using the data transfer communications that exists between neighbouring PEs. Generating one bit of a keypoint's descriptor consists of the following steps, also illustrated in Figure 3.

1. **Shift Analogue Data:** Shift copies of the gray-scale image according to the sampling pattern, bringing together pairs of pixel data to compare within each PE.
2. **Pixel Comparison:** Compare the pixel pairs' intensity values by subtracting one from the other.
3. **Binarize:** Examining the sign of the subtraction's result and from it generate a 1 bit binary value.

These steps are also outlined in Algorithm 3 of the supplementary. In order to generate all bits of the descriptor, these steps are performed repeatedly, for each pair of pixels from the sampling pattern. Note that the SIMD operation of SCAMP means these steps are executed across all PEs in parallel. Or put another way, descriptors are generated for every pixel location simultaneously, however only descriptors at keypoint locations are stored for use.

3.1.1. Optimization Of Pixel Shifting

By far, the largest computational cost of descriptor generation is the data transfer operation required to bring together pairs of pixel data for comparison. However, the number of these operations can be greatly reduced by optimisations described in our supplementary and shown in Table 1.

Descriptor Size	Naive	Optimized	Reduction
64	1224	**632**	48%
128	2445	**1112**	55%
256	4852	**1902**	61%

Table 1. The number of analogue data transfer operations used in descriptor generation, which is greatly reduced with optimisation.

3.2. In-Pixel Feature Storage

The previous Section described how in-pixel processing is used to generate descriptors, one bit at a time, for every pixel location in parallel. We now describe how, during generation, keypoint descriptors are stored across the PE array.

Specifically, each keypoint is assigned one row of PEs, across which its descriptor will be stored. Each PE of this row stores one bit of the descriptor, occupying one digital register, with the PE in the i-th column storing the i-th bit. This approach allows for far longer feature descriptors than can fit in the limited memory of a single PE.

This storage process has two stages. First, before any computation of descriptor bits is performed, an **Off-Sensor Row Planner** is executed. This assigns each keypoint a unique row of PEs to store its descriptor within. Next iterative computation of keypoint descriptors, one bit at a time is performed, using **On-Sensor Descriptor Movement** to store these bits within each keypoint's assigned PE row. This iterative process is illustrated in Figure 4 and Algorithm 1. These two stages are now described in more detail.

- **Off-Sensor Row Planner:** This stage first reads out the locations of all detected keypoints, from sensor to microcontroller. Using this information, a microcontroller program determines a unique row to assign to each keypoint for its descriptor storage. This mapping between

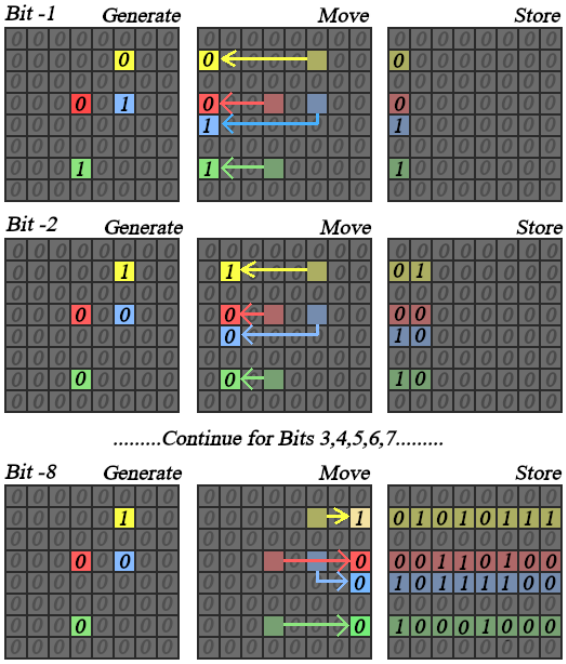

Figure 4. Illustration of keypoint descriptor generation and storage. Each grid cell represents a single PE, showing an 8×8 array for simplicity. Each detected keypoint (shown by each colour) is located inside a specific PE, and stores its descriptor into a specific row of PEs. From Top to Bottom, descriptors are generated one bit at a time and stacked horizontally, until complete descriptors are formed. At each step bits are generated (Left Column), moved to the correct PE for storage (Middle Column), and added alongside previously generated bits (Right Column). Examples of real features stored using this approach on SCAMP are in Figure 5.

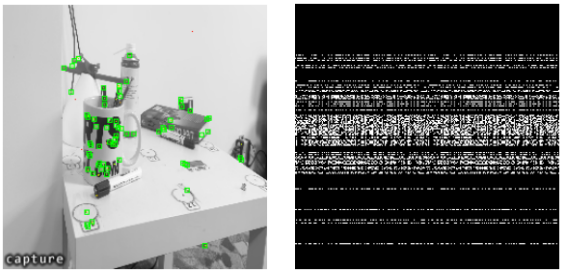

Figure 5. Left: Captured image and extracted keypoints from the FAST algorithm. Right: 256 bit descriptors generated for each keypoint. Each descriptor is stored across an entire row of PEs.

keypoints and rows is then stored for later use. More information on this aspect is included in supplementary.

- **On-Sensor Descriptor Movement:** Keypoint descriptors are generated in parallel, one bit at a time, as described in Section 3.1. After each set of bits is generated, they are moved to the PE at the correct row and column for storage. This involves first shifting each generated bit vertically into the row assigned to its keypoint by the row planner. Next, each bit is moved horizontally, such that the i-th bit of a descriptor is moved to the i-th column. As more bits are generated, this process stacks them horizontally, until each keypoint's row holds a complete descriptor, as illustrated in Figure 4. In practice, these bit movements can be performed for all keypoints in parallel, and accelerated by exploiting the SCAMP's digital flooding functionality.

Algorithm 1 Binary Descriptor Generation and Storage

```
Readout locations of keypoints;
Execute Off-Sensor Row Planner;
while i < Descriptor_Length do
    Generate i^th bits of keypoint descriptors;
    Move i^th bits of keypoint descriptors into correct
    PEs for storage (illustrated in Figure 4);
    i ↦ i + 1;
end while
```

Up to 256 descriptors, 256-bit in length, can be stored upon SCAMP's PE array (256×256) simultaneously. This approach also allows descriptors to be compared and matched entirely in-pixel, as described in the next section.

4. Binary Descriptor Matching

For each new captured frame, keypoints are detected and then binary descriptors are generated for these keypoints as described in Sections 3.1 and 3.2. We can also permanently store the keypoints and descriptors of a particular frame (a 'keyframe'), using the same layout. This allows us to then match any future keypoints against those of this keyframe. Our approach performs such keypoint descriptor matching entirely within the PE array, such that neither images or descriptors ever leave the SCAMP sensor. Such descriptor matching first involves a matching stage, generating a binary result within each PE. Then a "summation" of these bit matching results is performed, generating matching strength value for each descriptor. Finally, the descriptor with the strongest match is identified. Figure 6 illustrates this whole process, which is now described in more detail.

4.1. Descriptor Bit Matching

Matching a specific descriptor first involves replicating it across all rows of the PE array. Each row of PEs then holds a copy of the descriptor being matched, and a descriptor from the keyframe, if one is present in that row. The individual bits of both replicated and keyframe descriptors are aligned, with the i-th bits of each located in the same PEs, enabling direct bitwise comparison between descriptors. A bit matching routine then generates 1's in PEs where the descriptor bits match, and 0's otherwise. The number of bits matching within each row must then be evaluated, with the row with the greatest number of matching bits then in-

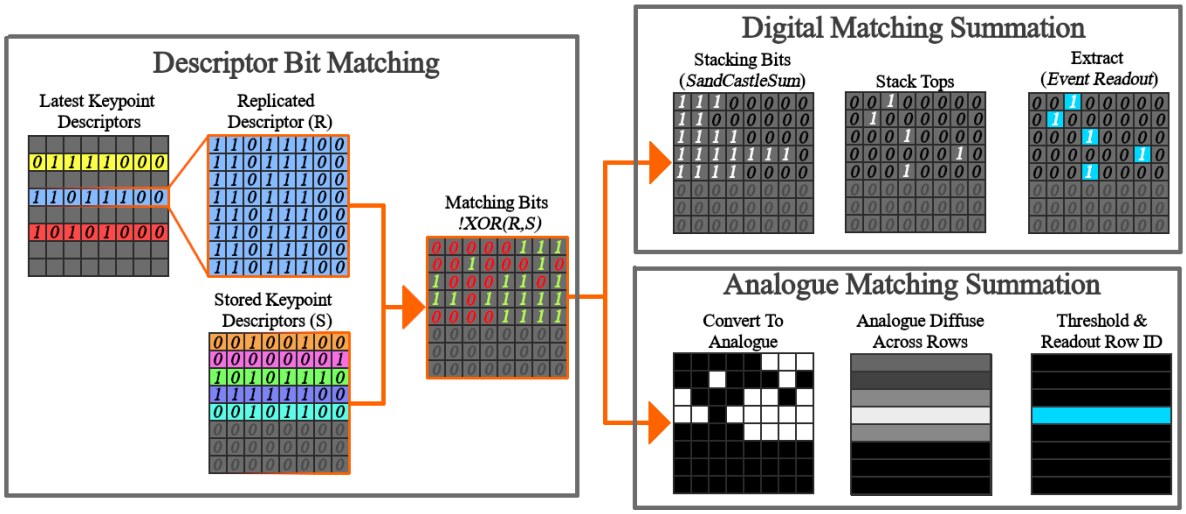

Figure 6. Illustration of the in-pixel descriptor matching process, shown on an 8×8 PE array for simplicity. From left to right, a keypoint descriptor is replicated across all rows of the PE array. Bit matching between the replicated and any stored keyframe descriptors within each row can then be performed in parallel. The next step is to determine which of the rows has the greatest number of matching bits, and hence which stored descriptor best matches the replicated one. There are two methods by which this may be achieved. **Digital** where the matching bits are stacked, and the longest pile is identified. **Analogue** where the bit matching result is converted to analogue, and then each descriptor's data is averaged by diffusion across its row of PEs.

dicating the best matching keyframe descriptor. We implemented two different methods to achieve this "summation" of matching bits, one digital, one analogue.

4.2. Bit Matching Summation : Digital

The *SandCastleSum* algorithm [3] is an efficient method of digital register plane summation. Such a register plane can be viewed as a binary image, with each PE holding one pixel, and this summation then computing the total number of white pixels or '1's in the image. This is accomplished by stacking '1's into piles, extracting the length of each pile, and adding these lengths together to find the total number of '1's. In our work we perform the stacking from *SandCastleSum* upon the results of the bit matching step, stacking matched bits across each row as illustrated in Figure 6. For scoring smaller descriptors of 128 and 64 bit, we created *MultiSandCastleSum*. This splits the digital register plane into several equally sized areas, each of which performs bit stacking independently as shown in Figure 7. This enables scoring of several smaller descriptors per row simultaneously.

After performing *MultiSandcastleSum*, the coordinates of the rightmost pixel within each *area* are readout. These pixels are those associated with the strongest matching descriptor from each *area*, their row coordinates identifying their associated stored descriptors, and column coordinates giving their matching strength. This is the only data read out required for each descriptor matching.

Two simple algorithms *Max ratio* and *Cross check*, are used to filter out incorrect matches between two descriptors' sets. *Max ratio* is the maximum ratio of the distance between the first and second closest descriptors' matching, and can be performed by reading out additional pixel coordinates (of the next best match) to calculate the ratio. *Cross check* only passes the matching when descriptor A in the current frame is the best match for descriptor B in storage, and descriptor B in storage is also the best match for de-

Algorithm 2 Binary descriptor matching on PPA

1: Replicate descriptor being matched;
2: Perform Descriptor Bit Matching (Figure 6 left)
3: **if** $Digital\ Scoring$ **then**
4: Score Bit Matching with $MultiSandCastleSum$; (Figure 6 Top Right)
5: **else**
6: Score Bit Matching with $Analogue\ Diffuse$ (Figure 6 Bottom Right)
7: **end if**
8: **for** $target_area$ in all $areas$ **do**
9: Read out $event$ from $target_area$;
10: Get keypoint & matching strength from $event$
11: Add matched keypoint into $Match\ result$;
12: **end for**
13: **if** $perform_cross_check_test$ **then**
14: Repeat step 1 to 12 for matched descriptor, against descriptors from latest frame;
15: **end if**

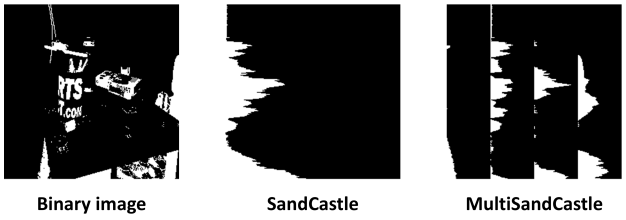

Figure 7. Visualization of the difference between SandCastle and MultiSandCastle of a binary image.

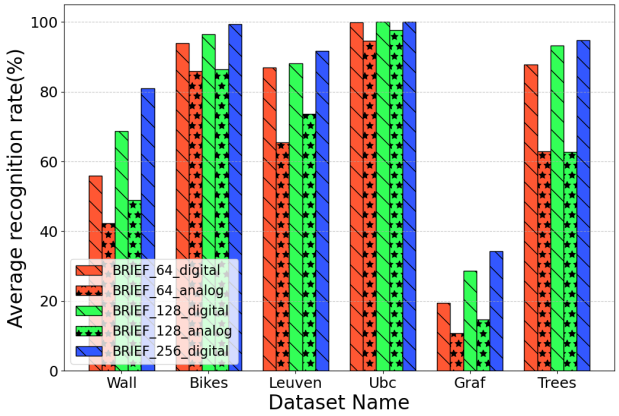

Figure 8. Recognition rates on six image sequences (illumination change: Leuven, viewpoint change: Graf and Wall, Image blur: Bikes and Trees, Image compression: Ubc). Shows results from various descriptor sizes (64, 128, 256), and from both methods of matching bit summation (digital, analogue).

scriptor A in the current frame. On PPA, *Cross check* repeats the matching process for the descriptors in storage.

4.3. Descriptor Matching Summation: Analogue

We also introduce a method of finding the best descriptor matching based upon analogue computation available on the SCAMP PPA. This process involves three steps. First, an analogue representation of the descriptor bit matching result is generated. This involves each PE mapping its bit matching result (*i.e.* 1 and 0) to high and low analogue values (*i.e.* 127 and -127) in a separate analogue register. Next, the **Analogue Diffusion** is used to "blur" this analogue representation horizontally, until each PE in a row has the same value, the average of that row. The PE row with the highest average then corresponds to the row of the best matching descriptor. This analogue diffusion process is analogous to connecting together many buckets of water, such that water flows freely between them, and then allowing time to pass and the water level of all buckets becoming equal. Finally, specific thresholds are applied to the diffused data to identify this best matching row. However, this method currently only supports descriptors up to size 128 bits, due to hardware limitations in the SCAMP for the size of area upon which analogue diffusion can reliably operate.

5. Experiments

We evaluated our in-pixel feature matching with experiments on two datasets, the Oxford Affine Covariant Features dataset, which has been widely used in the visual descriptor literature as benchmark [16, 21], and the Oxford robotic car dataset [13, 14] (some examples are shown in Figure 9). For each of these we tested our implementation on the SCAMP hardware system, using BRIEF descriptors of various length (64, 128, 256), extracted from local patches around keypoints detected by FAST of size 33×33 pixels. In addition, we tested both the digital and analogue methods of descriptor matching summation, for each descriptor size, as described in Sections 4.2 and 4.3.

Keypoint Detection Method Influence. We utilize a PPA implementation of the FAST algorithm [8] for keypoint detection. However, this is very sensitive to image changes, and not reliable for repeatedly detecting the same keypoints in a given scene. Rather than keypoint detection, this work is specifically interested in evaluating the performance of our in-pixel descriptor matching. Thus, for the Oxford Affine Covariant Features dataset experiments we removed the influence of the keypoint detection algorithm. To do so, keypoints are extracted via FAST from only the first image in each set. This initial set of keypoint locations are then transformed to new locations for each of the other images, using the Hermitian matrices provided by the dataset for each image. We provide experiments including the influence of FAST, where keypoints are extracted separately from each image, in the supplementary material. For the Oxford Robotic Car dataset, we extract FAST points from all images and measure localisation accuracy from matches identified. We expect significant improvements over FAST in future PPA-based in-pixel keypoint detectors, which our approach should easily be able to make use of.

5.1. Oxford Affine Covariant Features dataset

This dataset is made of sets of six images, each from various scenes and captured under different changing conditions such as camera location, orientation, illumination, and image compression. We extract keypoints via FAST from the first image of each set, generate and store BRIEF descriptors from these keypoints, and then attempt to match these descriptors against those of keypoints from the other five images. Some image sets were not used, such as those involving 90 degree image rotations, as the BRIEF descriptor is not scale or rotationally invariant.

Results. Figure 8 showcases the accuracy of keypoint matching using our in-pixel descriptors. As expected, in more challenging cases such as large viewpoint changes, longer descriptors result in greater accuracy. Additionally, using the analogue bit summation method results in lower accuracy due to the noise inherent to analogue operations.

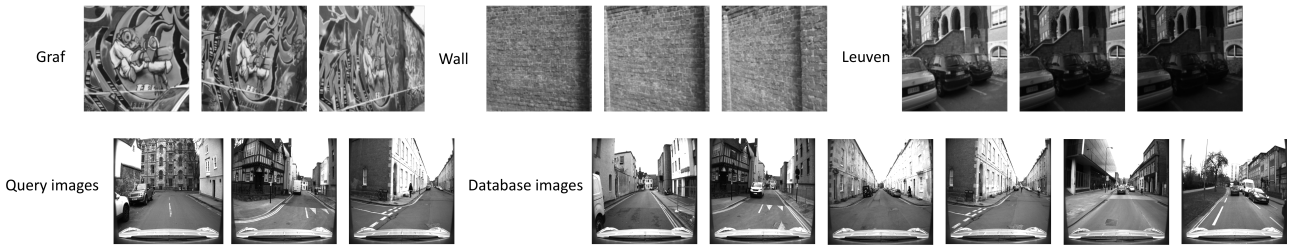

Figure 9. Examples of Oxford Affine Covariant Features dataset (top) and Oxford Robotic Car dataset (bottom).

5.2. Accuracy vs. Noise on PPA

Analogue operations on the SCAMP PPA always inject some measure of noise into their results. This affects both the descriptor generation, and the analogue bit summation for matching binary descriptors. To assess the impact of this noise, the BRIEF algorithm was run on a standard computer for comparison. The results in Table 2 present the average and standard deviation over five runs with varying keypoints and point pairs, calculated under both digital and analogue bit summation methods. Analogue noise has a relatively minor impact on performance when using the digital bit summation, excluding the illumination change (Leuven). In contrast, noise largely influences the accuracy when using the analogue bit summation.

5.3. Assesing Memory, Bandwidth & Speed

Memory. The on-sensor feature storage (in bits) for an image is calculated by multiplying the number of keypoints by the length of each descriptor.

Bandwidth. Table 4 shows the difference in the readout data rate required for three different approach paradigms. Our system's data readout and bandwidth use is composed of two parts: the readout of keypoint positions and the results of keypoint descriptor matching. Both of these involve reading out pixel coordinates, or *events*, each of which is two bytes in size. Keypoint positions are 1 *event* each, with the first byte being the x-coordinate and the second the y-coordinate. The descriptor matching results are also 1 *event* per matching, with the first byte being the best matching score, and the second byte the matching ID. As mentioned in Section 4.2, eliminating the low-confidence matches, *Max ratio* and *Cross check* require extra data readout. *Max ratio* requires an additional *event* per keypoint match (with the first byte of the event being the second best matching score), while the *Cross check* also requires 1 *event* per keypoint match (the first byte represents the best matching score, and the second the matching ID). The readout data rate of our system is therefore 4 *event*s (8 bytes) per keypoint, which is much lower than transferring a full gray-level image (65,536 bytes) and the descriptors (2N bytes per keypoint, where N is the size of descriptor). In a scenario when 100 keypoints are detected, the data rate reduction is over 80× compared to transmitting the full gray-level image. This increases to over 800× when only 10 keypoints are detected.

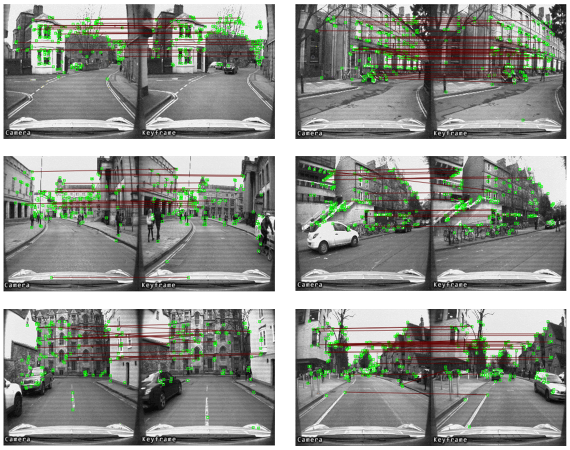

Figure 10. Examples of Matches in Oxford Robotic Car dataset.

Speed. The total time cost per frame includes data transmission (around 100 µs for 50 points) and sensor-level computations. Table 3 shows that our system implemented on the SCAMP camera achieves 209 fps with the digital scoring method, processing 50 keypoints (64-bit descriptor) per frame using the $Max\ ratio$. As the descriptor size increases to 128 and 256 bits, the frame rate decreases to approximately 91 fps and 50 fps respectively. In contrast, for the analogue scoring method, the system reaches 291 fps while scoring 50 keypoints (64-bit descriptors) with a one-round threshold decision. Notably, when scoring 128-bit descriptors, the frame rate only reduces to 210 fps.

5.4. Re-localization on Oxford Robotic Car Dataset

The Oxford Robotic Car Dataset contains image sequences and GPS location data along a 9-kilometer city route, recorded on different days and weather conditions. Correct image pairs are selected from different image sequences based on the GPS coordinates and chronological order of the images. Our system is evaluated on two tasks, *Img2Img* and *Seq2Seq*. In the *Img2Img* task, the query image is matched with each image in the dataset, and the image with

Sequence	Device	1|2		1|3		1|4		1|5		1|6	
		Digital	Analogue	Digital	Analogue	Digital	Analogue	Digital	Analogue	Digital	Analogue
Wall	PPA	94.0 ± 1.0	78.5 ± 2.7	90.4 ± 1.0	71.0 ± 5.5	81.7 ± 2.9	45.3 ± 3.3	55.1 ± 2.7	30.9 ± 3.3	23.0 ± 5.8	19.1 ± 2.0
	PC	100.0 ± 0.0	93.4 ± 3.3	94.2 ± 0.4	88.9 ± 2.5	83.7 ± 2.0	64.2 ± 5.2	60.6 ± 1.6	37.2 ± 5.3	27.4 ± 1.8	21.5 ± 1.3
Bikes	PPA	98.5 ± 1.4	90.5 ± 2.5	96.3 ± 1.0	86.8 ± 2.1	95.8 ± 1.3	85.2 ± 2.4	96.5 ± 0.6	83.6 ± 3.0	95.2 ± 1.9	85.7 ± 3.5
	PC	97.8 ± 0.6	96.1 ± 1.2	98.0 ± 0.4	96.1 ± 0.6	98.6 ± 0.3	92.4 ± 1.4	96.3 ± 0.3	92.1 ± 2.4	94.8 ± 0.9	89.4 ± 2.6
Leuven	PPA	95.4 ± 1.0	88.0 ± 1.7	95.1 ± 1.8	79.2 ± 2.4	85.7 ± 2.9	71.1 ± 5.3	86.1 ± 3.4	64.3 ± 1.9	78.5 ± 3.8	65.5 ± 2.7
	PC	100.0 ± 0.0	96.2 ± 1.0	98.5 ± 0.2	95.1 ± 1.5	98.1 ± 0.2	91.5 ± 1.4	97.4 ± 0.6	89.5 ± 3.4	99.4 ± 0.0	92.8 ± 1.6
Ubc	PPA	100.0 ± 0.0	98.8 ± 0.5	100.0 ± 0.0	98.6 ± 0.9	100.0 ± 0.0	98.7 ± 0.7	100.0 ± 0.0	97.7 ± 1.0	100.0 ± 0.0	94.2 ± 2.0
	PC	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	99.4 ± 0.4	100.0 ± 0.0	99.1 ± 0.5	100.0 ± 0.0	96.9 ± 0.7
Graf	PPA	52.7 ± 4.3	29.7 ± 3.4	32.3 ± 4.9	18.7 ± 2.0	8.6 ± 0.8	8.4 ± 2.4	10.9 ± 3.5	9.2 ± 1.8	8.0 ± 1.3	7.2 ± 1.3
	PC	55.2 ± 1.7	35.9 ± 3.0	34.4 ± 1.4	19.2 ± 3.9	12.7 ± 0.5	11.8 ± 3.1	15.6 ± 1.2	13.2 ± 2.3	9.9 ± 0.7	8.7 ± 1.4
Trees	PPA	91.6 ± 2.3	74.1 ± 2.8	90.3 ± 3.4	63.4 ± 3.5	81.4 ± 4.3	57.8 ± 3.2	84.9 ± 3.4	63.1 ± 2.7	83.5 ± 2.4	57.0 ± 3.0
	PC	93.8 ± 1.2	81.7 ± 2.2	91.9 ± 1.5	78.4 ± 4.3	87.7 ± 1.8	68.9 ± 4.6	89.5 ± 1.6	72.8 ± 3.2	84.4 ± 2.0	70.1 ± 3.1

Table 2. Recognition rates of Oxford Affine Covariant Features dataset (six image sequences) tested on the PPA and a standard computer (PC). The 128-bit BRIEF descriptor, extracted from 33×33 image patch, is deployed on the PPA and the PC for the test separately.

BRIEF size (Bit)	Feature management (µs)		Digital feature matching (µs)		Analogue feature matching (µs)					
					Threshold			Cross check		
	Generation	Storage	Max ratio	Cross check	iter 1	iter 2	iter 3	iter 1	iter 2	iter 3
64	705	828	53	51	28	48	57	26	48	56
128	1145	1609	152	149	30	56	66	29	56	64
256	1923	3223	285	281	/	/	/	/	/	/

Table 3. Execution time (µs) of our system. In feature matching, this table records the execution time (µs) per keypoint.

Readout	Size (bytes)	Transfer Time (µs)
Gray image	65536	16500
N-bit Descriptor	2N*	0.5N*
Our system	8*	2*

Table 4. The data readout required for keypoint descriptor matching under different approaches. * per keypoint matched.

BRIEF Size	Cross Check	Img2Img		Seq2Seq	
		Analogue	Digital	Analogue	Digital
64	False	78.6	90.9	96.8	98.3
	True	83.6	91.4	97.5	98.2
128	False	94.4	97.8	99.3	99.9
	True	94.5	98.1	99.5	99.9
256	False	/	98.5	/	100.0
	True	/	99.2	/	100.0

Table 5. Re-localization accuracy on Oxford robotic car dataset (For Seq2Seq matching, sequence length is 3 frames).

the highest number of matched keypoints is selected as the best match (See Fig. 10). In addition, a threshold (85% of the descriptor length) is used to validate the feature matches. A feature match is considered correct if the score exceeds this threshold, ensuring reliable matches. For the *Seq2Seq* task, the matching strategy follows the same procedure as that in *Img2Img*. See Table 5 for results. This test under realistic conditions demonstrates the applicability of in-sensor matching for localisation.

Please see the supplementary material for additional method details, experiments, videos, and results.

6. Conclusion and Future work

This work presents a novel *in-pixel* algorithm for the storage and matching of binary descriptors on PPAs. We make use of longer descriptors than any prior works, enabling matching of keypoints under significant image variations. As our approach performs all descriptor matching *in-pixel*, no images or descriptors ever leave the sensor plane. Instead, sensor output is reduced to just four high-level *event*s per key point match, providing a substantial ($> 100\times$) reduction in data transfer and bandwidth usage between sensor and processor. This removes the image transfer bottleneck, allowing operating speeds of up to ∼210fps, reducing power consumption, and making our approach inherently privacy enabling. Our approach, implemented upon the SCAMP PPA, was evaluated across a range of datasets with keypoint matching accuracies up to ∼94.5%. The FAST keypoint detector utilized in this work has significant stability issues. We envision improved keypoint detectors e.g [4], specifically designed for PPAs, will greatly enhance our approaches' real-world performance. Further, we believe this work paves the way for future in-pixel feature-based methods, for embedding applications such as mapping, re-localization, object and activity detection into the focal plane, for enhancing privacy, and for significantly reducing power and bandwidth consumption.

References

[1] Scamp5d vision system. https://scamp.gitlab.io/scamp5d_doc/. 2

[2] Laurie Bose, Jianing Chen, Stephen J. Carey, Piotr Dudek, and W. Mayol-Cuevas. Visual odometry for pixel processor arrays. *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 4614–4622, 2017. 2

[3] Laurie Bose, Piotr Dudek, Jianing Chen, and Stephen J. Carey. Sand castle summation for pixel processor arrays. *2021 17th International Workshop on Cellular Nanoscale Networks and their Applications (CNNA)*, pages 1–4, 2021. 5

[4] Laurie Bose, Piotr Dudek, and Jianing Chen. Descriptor-in-pixel : Point-feature tracking for pixel processor arrays. In *CVPR*, 2025. 8

[5] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal V. Fua. Brief: Binary robust independent elementary features. In *European Conference on Computer Vision*, 2010. 3

[6] Stephen J Carey, David RW Barr, and Piotr Dudek. Low power high-performance smart camera system based on scamp vision sensor. *Journal of Systems Architecture*, 59 (10):889–899, 2013. 2

[7] Hector Castillo-Elizalde, Yanan Liu, Laurie Bose, and W. Mayol-Cuevas. Weighted node mapping and localisation on a pixel processor array. *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6702–6708, 2021. 2, 5

[8] Jianing Chen, Stephen J. Carey, and Piotr Dudek. Feature extraction using a portable vision system. In *Workshop on Vision-based Agile Autonomous Navigation of UAVs, IROS*, 2017. 2, 6

[9] Piotr Dudek, Thomas S. Richardson, Laurie Bose, Stephen J. Carey, Jianing Chen, Colin Greatwood, Yanan Liu, and W. Mayol-Cuevas. Sensor-level computer vision with pixel processor arrays for agile robots. *Science robotics*, 7 67: eabl7755, 2022. 1

[10] Mathias Gehrig and Davide Scaramuzza. Recurrent vision transformers for object detection with event cameras. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13884–13893, 2022. 2

[11] Javier Hidalgo-Carri'o, Guillermo Gallego, and Davide Scaramuzza. Event-aided direct sparse odometry. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5771–5780, 2022.

[12] Siqi Li, Zhikuan Zhou, Zhou Xue, Yipeng Li, Shaoyi Du, and Yue Gao. 3d feature tracking via event camera. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18974–18983, 2024. 2

[13] Will Maddern, Geoff Pascoe, Chris Linegar, and Paul Newman. 1 Year, 1000km: The Oxford RobotCar Dataset. *The International Journal of Robotics Research (IJRR)*, 36(1):3–15, 2017. 2, 6

[14] Will Maddern, Geoffrey Pascoe, Matthew Gadd, Dan Barnes, Brian Yeomans, and Paul Newman. Real-time kinematic ground truth for the oxford robotcar dataset. *arXiv preprint arXiv: 2002.10152*, 2020. 2, 6

[15] Nico Messikommer, Carter Fang, Mathias Gehrig, and Davide Scaramuzza. Data-driven feature tracking for event cameras. *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5642–5651, 2022. 2

[16] Krystian Mikolajczyk and Cordelia Schmid. A performance evaluation of local descriptors. *2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 2003. Proceedings.*, 2:II–II, 2003. 2, 6

[17] Raul Mur-Artal, José M. M. Montiel, and Juan D. Tardós. Orb-slam: A versatile and accurate monocular slam system. *IEEE Transactions on Robotics*, 31:1147–1163, 2015. 2

[18] Riku Murai, Sajad Saeedi, and Paul H. J. Kelly. Bit-vo: Visual odometry at 300 fps using binary features from the focal plane. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 8579–8586, 2020. 2

[19] Hongwei Ren, Jiadong Zhu, Yue Zhou, Haotian Fu, Yulong Huang, and Bo-Xun Cheng. A simple and effective point-based network for event camera 6-dofs pose relocalization. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18112–18121, 2024. 2

[20] Sachin Shah, Matthew Albert Chan, Haoming Cai, Jingxi Chen, Sakshum Kulshrestha, Chahat Deep Singh, Yiannis Aloimonos, and Christopher A. Metzler. Codedevents: Optimal point-spread-function engineering for 3d-tracking with event cameras. *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 25265–25275, 2024. 2

[21] Tinne Tuytelaars and Krystian Mikolajczyk. Local invariant feature detectors: A survey. *Foundations and Trends® in Computer Graphics and Vision*, 3(3):177–280, 2008. 2, 6